YOLOv8 for Web UI Element Detection: A CNN Ablation Study

Alden Bernstein [1]

Abstract

We apply YOLOv8, a convolutional neural network (CNN) for object detection, to the task of localizing UI elements in web page screenshots. We crawl ∼260K screenshots from the Tranco Top 1M websites and conduct **14 controlled ablation experiments** covering (a) architectures (YOLOv8n/s/m and a custom model), (b) network depth, (c) optimizers (SGD vs. AdamW), (d) pooling (max vs. average), and (e) activation functions (SiLU vs. ReLU), plus learning rate, resolution, augmentation, and initialization. We find that architecture produces the largest single-factor gains, while other hyperparameters yield smaller but consistent effects. The ablation's per-class analysis also revealed that 3 of 5 classes were undetectable regardless of configuration, leading us to restructure the task into single-class detection with deduplication and diverse sampling—improving mAP50 from 0.136 to **0.323** (2.4×) using the same model. This supports OmniParser's (Lu et al., 2024) single-class approach on independent data. Development of the report and scripts was assisted by Claude Code.

1. Introduction

For this project we wanted to apply CNN-based object detection to a non-standard dataset rather than a typical benchmark like CIFAR-10 or ImageNet. We chose web UI element detection—finding buttons, links, inputs, and other interactive elements in screenshots of websites—because (1) we could crawl our own training data at scale, and (2) OmniParser (Lu et al., 2024) recently showed that YOLOv8 works well for this task, giving us a baseline to replicate and extend.

The practical motivation is browser agent security: LLM-powered agents (browser-use, 2024) that navigate websites can be tricked by hidden elements in the DOM (Greshake et al., 2023). A CNN that detects elements from the rendered screenshot only sees what is actually visible, naturally filtering hidden attacks. But for this report, our main focus is the CNN ablation study itself—systematically testing how different hyperparameter choices affect detection performance, as we did with simpler CNNs in the homework assignments.

We conduct 14 experiments varying one factor at a time, covering all five hyperparameter categories from the project guidelines: (a) architectures, (b) number of layers, (c) optimizers, (d) pooling functions, and (e) activation functions. Based on the per-class results from the ablation, we iterated on the data pipeline (16 classes → 5 classes → 1 class with deduplication and diverse sampling), achieving a 2.4× improvement following insights from the ablation.

2. Related Work

CNN architectures have been a focus throughout this course. ResNet (He et al., 2016) showed that skip connections enable training very deep networks. VGG (Simonyan & Zisserman, 2015) demonstrated that stacking small 3×3 filters works better than larger kernels. MobileNet (Sandler et al., 2018) introduced depthwise separable convolutions for efficient mobile inference. DenseNet (Huang et al., 2017) connected every layer to every other layer for feature reuse. These ideas—depth, filter design, efficiency, connectivity—all show up in YOLOv8's architecture.

YOLOv8 (Ultralytics, 2023) is a single-stage object detector that predicts bounding boxes and class labels in one forward pass. It uses a CSPDarknet53 backbone with C2f blocks (a form of residual connection), a Feature Pyramid Network neck for multi-scale detection, and Spatial Pyramid Pooling (SPPF) for capturing context at multiple scales. We chose it because OmniParser (Lu et al., 2024) showed it works for UI detection, and because it lets us ablate the same CNN components we studied in class (pooling, activation functions, depth, etc.).

OmniParser (Lu et al., 2024; Microsoft, 2025) fine-tunes a YOLO model on UI screenshots for screen parsing, using a single "icon" class. They combine this with OCR and captioning for a full pipeline. We replicate the single-class detection approach on our own independently crawled dataset and provide a systematic ablation that OmniParser did not include.

3. Method

3.1. Dataset

We built a Playwright-based web crawler that visits websites from the Tranco Top 1M ranking, takes screenshots at 1920×1080 (top, mid, and bottom viewport sections), and extracts bounding box annotations from DOM element positions. This produced ∼260K annotated images. A separate pre-existing set of 668 annotated screenshots is used for validation and test splits, ensuring zero overlap with crawled training data.

Class refinement (16 → 5 → 1). The crawler initially produced 16 fine-grained classes (link, button, input, checkbox, etc.). We found severe class imbalance—the top 3 classes comprised 66% of all annotations while the bottom 6 contributed <1%. We first merged these into 5 groups for the ablation study, but per-class results showed that only 2 of 5 groups achieved meaningful detection (Table 2). Following OmniParser's approach (Lu et al., 2024), we then consolidated everything into a single `ui_element` class, removed duplicate and tiny bounding boxes, and selected 1 image per domain for diversity (∼47K training images). This iterative restructuring is described in Section 4.4.

3.2. CNN Architecture: YOLOv8

YOLOv8 is a fully convolutional network with three parts, each containing components we ablate:

Backbone (CSPDarknet53). Convolutional layers with Cross-Stage Partial connections extract features at multiple scales through stride-2 downsampling and **C2f blocks**—residual bottleneck modules that work like stacking conv layers but with a shortcut that preserves the original features (similar to ResNet skip connections). Their repeat count is controlled by a *depth multiplier*. All convolutions use **SiLU** activation by default ($x \cdot \sigma(x)$), which unlike ReLU ($\max(0, x)$) allows small negative gradients through, reducing the dead neuron problem.

Neck (Feature Pyramid Network). Multi-scale features are fused via upsampling and concatenation, allowing the detector to combine fine-grained spatial detail from earlier layers with semantic information from deeper layers.

Head + SPPF. Detection heads predict bounding boxes and class probabilities at three scales in a single forward pass (unlike image classification which outputs one label per image, detection outputs multiple boxes per image). The **SPPF module** applies cascaded `MaxPool2d` at different kernel sizes to capture both local details and global context—we test `AvgPool2d` as an alternative, similar to the pooling comparison between max and average pooling in standard CNNs.

The model family spans nano (3.2M params), small (11.2M), and medium (25.9M) variants.

3.3. Training Setup

Hardware. All training on an NVIDIA GPU.

Baseline. YOLOv8n (nano), 640×640, SGD, lr=0.01, 25 epochs, mosaic augmentation, SiLU, max pooling, COCO-pretrained, batch 16. For the 5-class ablation, we use 8.5% of data (∼22K images) to fit all 14 experiments in our compute budget. The 1-class experiment uses the full diverse subset (∼47K images).

Metrics. We report mAP50 (mean Average Precision at IoU≥0.5: a predicted box counts as correct if it overlaps at least 50% with the ground truth), mAP50-95 (averaged over stricter IoU thresholds), precision, recall, CPU latency (ms), and model size (MB).

4. Experiments

4.1. Ablation Design

We run 14 experiments, each varying exactly **one factor** from baseline. We cover all five CNN hyperparameter categories from the project guidelines:

(a) Architectures (3 experiments). YOLOv8n (baseline), YOLOv8s, YOLOv8m, plus a custom lightweight model (halved channels, 2-scale detection, 0.71 MB).

(b) Number of layers (1 experiment). Depth multiplier 0.33 → 0.67, doubling C2f block repeats from 10 to 20.

(c) Optimizer (1 experiment). SGD (baseline) vs. AdamW.

(d) Pooling (1 experiment). MaxPool2d (baseline) vs. AvgPool2d in the SPPF module.

(e) Activation function (1 experiment). SiLU (baseline) vs. ReLU across all convolutional layers.

Plus: learning rate (2), image resolution (1), augmentation (1), initialization (1), and best combination (1).

4.2. 5-Class Ablation Results

Table 1 shows all 14 experiments on the 5-class dataset.

4.3. Analysis of Hyperparameter Effects

(a) Architecture matters most. YOLOv8s (11.2M parameters) jumps to mAP50=0.134 from the near-zero YOLOv8n baseline (0.002, which reflects the severe class imbalance in the 5-class formulation). YOLOv8m (25.9M) does not improve further despite 2.3× more parameters—on limited data, bigger models hit diminishing returns fast.

(b) More layers do not help. Doubling C2f depth produces 0.107 mAP50—no better than the baseline-depth model

Table 1. 5-class ablation results (∼22K training images, 8.5% of dataset). Each experiment varies one factor from baseline. Bold = best per column.

Experiment	mAP50	mAP50-95	Prec.	Recall	CPU (ms)	Size (MB)
best_combo	**0.136**	**0.105**	0.431	**0.169**	183.7	21.5
arch_yolov8s	0.134	0.094	0.431	0.148	39.3	21.5
arch_yolov8m	0.128	0.094	**0.472**	0.122	106.5	49.6
imgsz_1280	0.126	0.090	0.213	0.152	64.0	6.0
aug_no_mosaic	0.116	0.076	0.408	0.128	18.2	5.9
lr_0.001	0.116	0.078	0.432	0.115	19.7	5.9
lr_0.005	0.116	0.078	0.432	0.115	19.4	5.9
optimizer_adamw	0.110	0.070	0.358	0.156	19.3	5.9
init_scratch	0.108	0.070	0.386	0.133	18.7	5.9
layers_deep	0.107	0.071	0.383	0.118	28.2	8.0
pooling_avg	0.106	0.067	0.412	0.109	18.7	5.9
activation_relu	0.103	0.064	0.358	0.131	19.3	5.9
arch_custom_cpu	0.077	0.042	0.357	0.106	**9.8**	**0.71**
baseline	0.002	0.001	0.002	0.005	1112	5.9

trained from scratch (0.108), but slower (28ms vs. 19ms CPU). Extra layers add cost without benefit on this dataset.

(c) AdamW vs. SGD. AdamW gets 0.110 vs. SGD's 0.108—a tiny difference compared to architecture effects. AdamW does give slightly better recall (0.156 vs. 0.133).

(d) Max pooling slightly outperforms average pooling. AvgPool2d in SPPF yields 0.106 vs. 0.108 with MaxPool2d. Max pooling preserves sharp features like edges of buttons and input fields. The difference is small (0.002 mAP50).

(e) SiLU slightly outperforms ReLU. Replacing SiLU with ReLU drops mAP50 from 0.108 to 0.103. SiLU's smooth gradient helps optimization—consistent with why modern architectures default to it over ReLU.

Best combo. Combining winners (YOLOv8s + AdamW + lr=0.001 + 1280px + no mosaic) gives only 0.136—just +0.002 over standalone YOLOv8s. Combining the best settings from each axis yields only marginal additional gains.

Per-class breakdown. Table 2 reveals why: only `interactive` (0.375) and `form_input` (0.288) classes are detectable. The other three classes are nearly zero regardless of hyperparameters, which led us to try consolidating to a single class.

4.4. Data Refinement: 1-Class Consolidation

Motivated by the ablation's per-class findings (Table 2) and OmniParser's single-class approach (Lu et al., 2024), we consolidated all annotations to one `ui_element` class and applied two cleaning steps:

1. **Deduplication and noise removal**: removed duplicate bounding boxes (same coordinates rounded to 3 decimals) and tiny boxes (<0.01% of image area).
2. **Diverse sampling**: selected 1 image per domain (preferring top-of-page views), producing ∼47K training images from ∼15K unique domains instead of multiple similar screenshots per site.

Table 3 shows the result. The same YOLOv8n model that scored 0.002 on 5-class data now achieves **0.323 mAP50** on 1-class cleaned data—a 2.4× improvement over our best 5-class experiment (YOLOv8s at 0.134) after reformulating the task and improving data quality. Recall jumps from 0.005 to 0.426.

Starting with 16 fine-grained classes was our initial mistake—the per-class analysis from the ablation experiments is what revealed this, and restructuring the task into a single detection class with diverse sampling fixed it.

5. Conclusion

We trained YOLOv8 for UI element detection in web screenshots, conducting 14 ablation experiments across all five standard CNN hyperparameter categories—architectures, layers, optimizers, pooling, and activations. The ablation identified real effects: architecture scale matters (YOLOv8s outperforms nano and medium), SiLU outperforms ReLU, and max pooling edges out average pooling. The per-class analysis also revealed that our initial 16-class taxonomy was too granular for the data we had—3 of 5 merged classes were undetectable regardless of hyperparameters. Restructuring into a single `ui_element` class with deduplication and diverse sampling brought mAP50 from 0.136 to 0.323, supporting OmniParser's single-class approach on independent data.

6. Bonus Points

(b) Large-scale data collection and iterative preparation. We built a custom Playwright-based web crawler

Table 2. Per-class AP50 for top 5-class models. Three of five classes are essentially undetectable.

Experiment	interactive	form_input	form_ctrl	text_lbl	media
best_combo	0.375	0.263	0.019	0.019	0.005
arch_yolov8s	0.375	0.288	0.000	0.005	0.001
arch_yolov8m	0.364	0.271	0.000	0.005	0.002
imgsz_1280	0.368	0.232	0.018	0.004	0.005

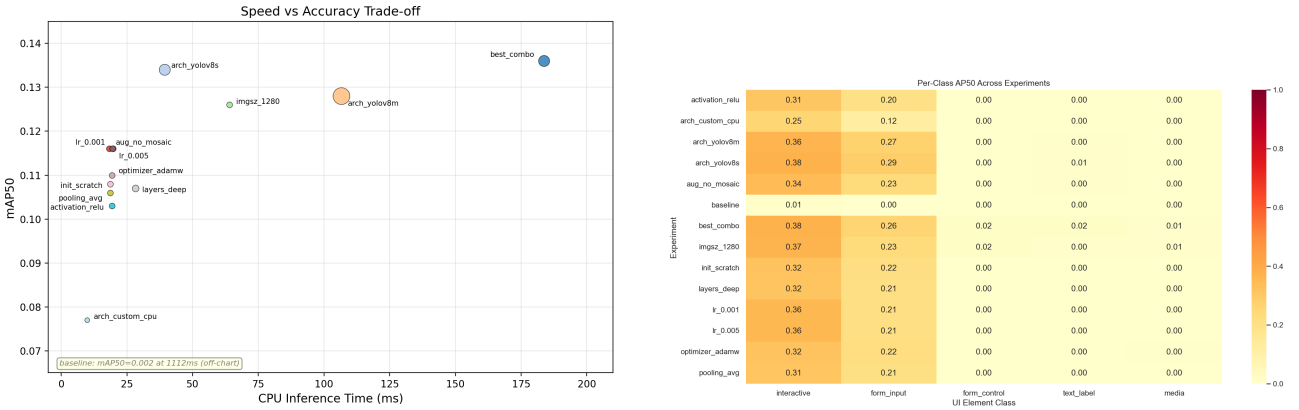

Figure 1. **Left:** Speed–accuracy trade-off across the 14 CNN ablation experiments. **Right:** Per-class AP50 heatmap showing that only `interactive` and `form_input` are consistently detected in the 5-class setting, motivating the 1-class consolidation.

Table 3. Impact of data refinement. Same YOLOv8n model, different data preparation.

Setup	mAP50	Recall	Images
5-cls, 8.5% (baseline)	0.002	0.005	22K
5-cls, 8.5% (best combo)	0.136	0.169	22K
1-cls, diverse	**0.323**	**0.426**	**47K**

that visits real websites from the Tranco Top 1M, captures multi-viewport screenshots, and extracts bounding box annotations from DOM element positions—producing ∼260K annotated training images. We then ran the full training pipeline multiple times: first with 16 classes, then with 5 merged classes, and finally with 1 consolidated class—each iteration informed by the previous round's per-class results. This iterative process of crawling, reorganizing, deduplicating, and retraining required significant effort beyond downloading an existing benchmark.

(a) Novel application: prompt injection classifier. While the ablation study focuses on visual detection, the failure modes we identify (e.g., hidden or off-screen DOM elements that LLM agents can still read) motivate complementary text-based filtering. As a preliminary extension, we trained a DeBERTa-v3-xsmall (He et al., 2021) text classifier (22M parameters) for detecting prompt injection attacks in UI element text. The model uses focal loss (Lin et al., 2017) ($\gamma = 2.0$, $\alpha = [0.7, 0.3]$)—a variant of cross-entropy that down-weights easy examples so the model focuses on hard cases. We trained on 23K balanced examples from six public sources and ran 17 ablation experiments across architecture, loss function, class weighting, sequence length, and data composition. Table 4 compares against two published baselines on independent benchmarks with zero training data overlap.

Table 4. Text classifier on independent benchmarks. Our 22M-parameter model achieves competitive performance against larger baselines.

Benchmark	Metric	Ours (22M)	ProtectAI (200M)	Meta (86M)
NotInject EN (253)	Acc	**89.3**	62.5	0.0
	FPR	**10.7**	37.5	100.0
Wildguard (971)	Acc	**97.8**	75.2	6.7
	FPR	**2.2**	24.8	93.3
Gandalf (112)	Recall	92.0	**100.0**	**100.0**
InjecGuard (144)	Acc	**71.5**	65.3	43.8

For deployment, we exported to ONNX with INT8 quantization, reducing latency from 16.1ms to 3.8ms (4.2× speedup) and model size from 271 MB to 83 MB. We include this as a preliminary exploration rather than a fully integrated system.

Development of the report and scripts was assisted by Claude Code.

References

browser-use. browser-use: Make websites accessible for ai agents. https://github.com/browser-use/browser-use, 2024.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

He, P., Gao, J., and Chen, W. Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*, 2021.

Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4700–4708, 2017.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.

Lu, Y., Xu, J., Zhang, Y., Zhao, C., He, J., Wu, Y., Fan, L., Qiao, J., et al. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*, 2024.

Microsoft. Omniparser v2. https://github.com/microsoft/OmniParser, 2025.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2015.

Ultralytics. Yolov8. https://github.com/ultralytics/ultralytics, 2023.